

The Surface-Energy-Estimator

Julia Messthaler

Blazingly fast code for finding crystals (subsets of a graph) that minimize some quadratic form.

Subcommands

energies

The command `messthaler-wulff energies fcc -p 5 20` tries to find the optimal energies for crystals in the fcc lattice (with distances from the origin at most 5, hence `-p 5`) that also have at most 20 atoms.

More on specifying graphs

crystals

The command `messthaler-wulff crystals fcc -p 5 20 112` tries to find all crystals meeting the criteria in the previous section and having exactly 20 atoms and an energy of 112.

More on specifying graphs

stats

The command `messthaler-wulff stats fcc -p 5 -` reads a crystal from the console (like the ones that `messthaler-wulff crystals` outputs) and outputs the number of atoms in it and its energy.

More on specifying graphs

More on specifying crystals

plot

The command `messthaler-wulff plot fcc plc -` reads a crystal from the console and displays it using matplotlib. The value `plc` shows

- `p` - the points
- `l` - the edges
- `c` - the convex hull

This command can only render crystals in 2 or three dimensions. The dimension is detected automatically based on the bravais lattice passed in.

[More on specifying graphs](#)

[More on specifying crystals](#)

Specifying graphs

A graph consists of a type (which is either a bravais lattice or a finite graph) and some optional parameter. The optional parameter can and must only be specified when entering a bravais lattice as it is the radius of the generated graph. This program in its current state does not support infinite lattices.

The type can be one of

- A path to a file on your computer which contains the correct json code (more on that later)
- One of the preset values:
 - **square**
 - **cubic**
 - **triangular**
 - **fcc**
- The value `-`, which indicates that the value should be read in from the console

The parameter is simply specified using the optional `-p` flag.

A json example for a finite graph:

```
{
  "type": "finite",
  "nodes": [1,2,3,4,5, [1,2,3], [2,4], "Node"],
  "edges": [
    [1,2],
    [3,4],
    [1,4],
    [1,5],
    [5,2],
    [5,3],
    [[1,2,3],[2,4]],
    [5, "Node"]
  ]
}
```

As you can see, a node may be one of

- An integer
- A list of integers
- A string

A json example for a bravais lattice:

```
{
  "type": "bravais",
  "primitives": [
    [2,0],
    [0,2],
    [1,1]
  ]
}
```

The primitives are the values that generate the lattice, meaning the difference between a lattice point and one of its neighbors is exactly a primitive or minus a primitive.

Specifying crystals

Crystals are a lot simpler than graphs, as they are only a list of nodes, like this:

```
[
  [1,2,3],
  2,
  "Node"
]
```

This is a crystal with three atoms which can be viewed as a subgraph of the finite graph presented in the json example in the previous section.

Mathematical Background

This problem has two equivalent formulations: The graph based one and the sparse matrix based one. The graph based formulation lends itself very well to implementation and the matrix formulation makes calculations easy.

Let us discuss the matrix formulation. We have some sparse matrix $L \in \mathbb{R}^{n \times n}$ and we are trying to optimize the quadratic form $v^\top L v$ with $v \in \{0, 1\}^n$.

To do this we generate essentially random vectors and memorize the best value seen so far. The precise method for generation involves locally optimal transformations, let $R(v)/A(v)$ denote all crystals that can be reached by removing/adding an atom. Locally optimal removals/additions are the values of $R(v)/A(v)$ with optimal energy.

Calculating the new energy after such a transformation is also rather easy; let us calculate it for adding the atom e_i to the vector v :

$$\begin{aligned} \delta_i &:= (v + e_i)^\top L (v + e_i) - v^\top L v = e_i^\top L e_i + e_i^\top L v + v^\top L e_i \\ &= e_i^\top L e_i + 2e_i^\top L v \end{aligned}$$

This last equality holds if we assume that L is symmetric, which in this program is always the case, though the method would probably work just as well for non-symmetric matrices.

This is now where it helps to understand the graph formulation of this problem. We can represent our matrix L as a graph by creating n nodes and assigning each as a weight the i -th diagonal entry in the matrix. We then add an edge between the nodes i and j if $L_{i,j} \neq 0$ and give the edge as weight $L_{i,j}$. If L is symmetric, then we can view our graph as non-directed.

δ_i can now be seen as the weight of the i -th node plus twice the edge weights to its neighbors that are in the crystal.

In this program we also usually assume that off-diagonal entries are -1 or 0 , meaning that the only edge weight possible is -1 .

When generating random vectors we essentially perform a random walk. Currently we decide using a coin flip whether to remove or add an atom and then pick a random direction from the locally optimal removals/additions. This process yields very good results very quickly.

Future Work

It is fascinating why exactly this random process converges against optimal values and it would be very useful to have not just a mathematical understanding of it, but perhaps also some guarantees.

It would also be of interest if this method is general, as in if it manages to optimize arbitrary quadratic forms corresponding to arbitrary graphs or if instead it is only applicable to bravais lattices.

This algorithm is unfortunately only probabilistic and it would be useful to be able to compute exact solutions, but any innovation is stifled by a lack of mathematical understanding of the dynamics of the problem.